DuocUC® INFORMÁTICA Y TELECOMUNICACIONES

ReponseEntity y Validaciones

DESARROLLO FULLSTACK I

DSY1103

FORMAMOS PERSONAS PARA UNA SOCIEDAD MEJOR

• Contenidos

- Respuestas personalizadas con ResponseEntity
- Validaciones con Bean Validation (JSR 380)
- Desarrollo de actividades practicas.

01

ResponseEntity

• ¿Qué es ResponseEntity?

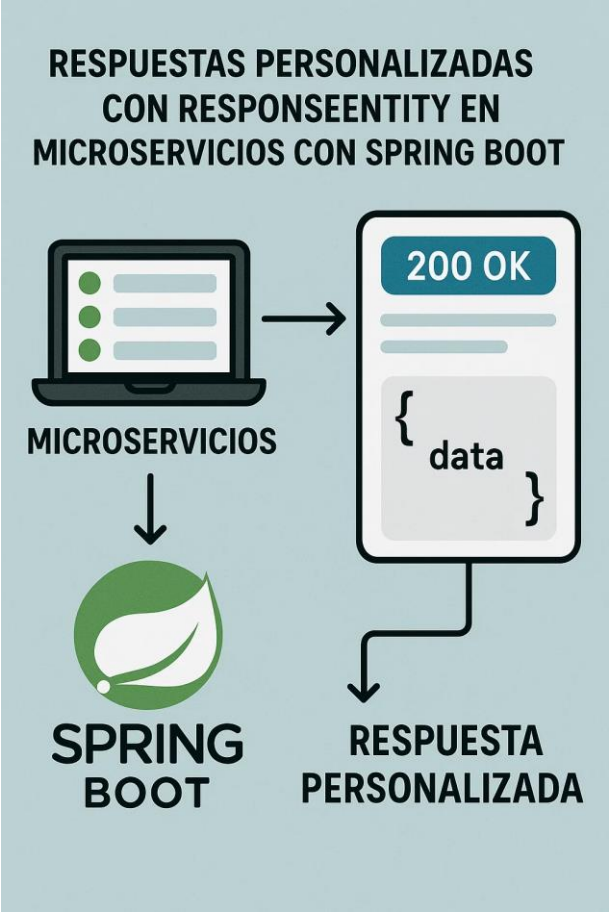
ResponseEntity es una clase de Spring Framework que representa la respuesta HTTP completa a enviar desde un controlador. Permite envolver el cuerpo de la respuesta junto con los detalles de la respuesta HTTP, como el código de estado (Status Code) y los encabezados (Headers).

Características Clave

- **Control Total del HTTP**: Permite establecer con precisión el Status Code (200, 201, 400, 404, 500, etc.), esencial para el correcto diseño RESTful.

- **Encabezados Personalizados**: Facilita la adición de información relevante en los encabezados, como Location (para recursos creados), Content-Type o tokens.

- **Flexibilidad**: Puede ser usado para devolver cualquier tipo de objeto en el cuerpo o solo el estado (ResponseEntity<Void>)



• ¿Qué es ResponseEntity?

El uso de **ResponseEntity** asegura que las aplicaciones cliente puedan interpretar correctamente el resultado de una operación

Estado HTTP	Definición	Uso en el Código
201 Created	Indica que se creó un nuevo recurso.	new ResponseEntity<>(obj, HttpStatus.CREATED)
204 No Content	Éxito en la acción sin cuerpo de respuesta.	ResponseEntity.noContent().build()
400 Bad Request	La petición del cliente es inválida (ej. validación fallida).	Se maneja automáticamente con @Valid y @ControllerAdvice.
404 Not Found	El recurso solicitado no existe.	ResponseEntity.notFound().build()

• ¿Qué es ResponseEntity?

Te permite pasar de:

```
@GetMapping("/productos")
public List<Producto> listar() {
    return service.findAll();
}
```

ResponseEntity

Video de Apoyo

a algo más profesional:

```
@GetMapping("/productos")
public ResponseEntity<List<Producto>> listar() {
    List<Producto> productos = service.findAll();
    return ResponseEntity.ok(productos); // 200 OK
}
```

• Buenas prácticas didácticas para ResponseEntity

1. **Siempre pensar primero el código HTTP correcto**, luego el body.
2. No devolver **200 OK** para todo “porque funciona”; usar:
   - **201 CREATED** para creaciones
   - **204 NO CONTENT** cuando no hay body
   - **400 y 422** para errores del cliente
   - **404** para recursos inexistentes
3. Crear un **DTO de error estándar** y SIEMPRE devolverlo mediante ResponseEntity.
4. No devolver entidades JPA crudas; usar DTOs para:
   - evitar exponer de más
   - independizar la API del modelo interno
5. Aprovechar los **headers**:
   - para paginación
   - para correlación (X-Correlation-Id)
6. Integrar **ResponseEntity** con:
   - Bean Validation
   - ControllerAdvice
   - Swagger/OpenAPI (mostrar responses 200/400/404/500 bien documentados).

02

Bean Validation

• Validaciones con Bean Validation (JSR 380)

Bean Validation es un estándar (JSR 380/Jakarta Bean Validation) para validar modelos de objetos Java. Define un conjunto de anotaciones estándar que se aplican a los campos de las clases (DTOs o Entidades).

Hibernate Validator es la implementación de referencia que utiliza Spring Boot, eliminando la necesidad de escribir lógica de validación manual

Anotación	Uso	Ejemplo
@NotNull	Asegura que el campo no es nulo.	@NotNull
@NotBlank	Asegura que la cadena no es nula, vacía, o solo espacios.	@NotBlank(message="...")
@Size	Define los límites de longitud para colecciones o cadenas.	@Size(min=5, max=50)
@Email	Valida que la cadena tiene formato de	@Email

• Validaciones con Bean Validation (JSR 380)

Características y Librerías

- **Librería Principal:** Hibernate Validator (incluida en spring-boot-starter-validation).
- **Estándar:** JSR 380 (o Jakarta Bean Validation).
- **Uso:** Anotaciones aplicadas a nivel de campo en la clase.

Integración en Spring Boot

1. **Anotar el DTO:** Usar las anotaciones en la clase de entrada (ProductoDto).
2. **Habilitar en el Controlador:** Usar la anotación @Valid en el parámetro @RequestBody del método del controlador para activar el proceso de validación.

```java
public class ProductoRequest {

    @NotBlank(message = "El nombre es obligatorio")
    @Size(max = 100, message = "El nombre no puede superar 100 caracteres")
    private String nombre;

    @NotNull(message = "El precio es obligatorio")
    @Positive(message = "El precio debe ser mayor a 0")
    private Double precio;

    @NotNull(message = "El stock es obligatorio")
    @PositiveOrZero(message = "El stock no puede ser negativo")
    private Integer stock;
}
```

```java
@PostMapping("/productos")
public ResponseEntity<Producto> crear(@Valid @RequestBody ProductoRequest request) {
    Producto producto = mapper.toEntity(request);
    Producto creado = service.save(producto);
    return ResponseEntity.status(HttpStatus.CREATED).body(creado);
}
```

Cuando algo no cumple las anotaciones, Spring lanza **MethodArgumentNotValidException**

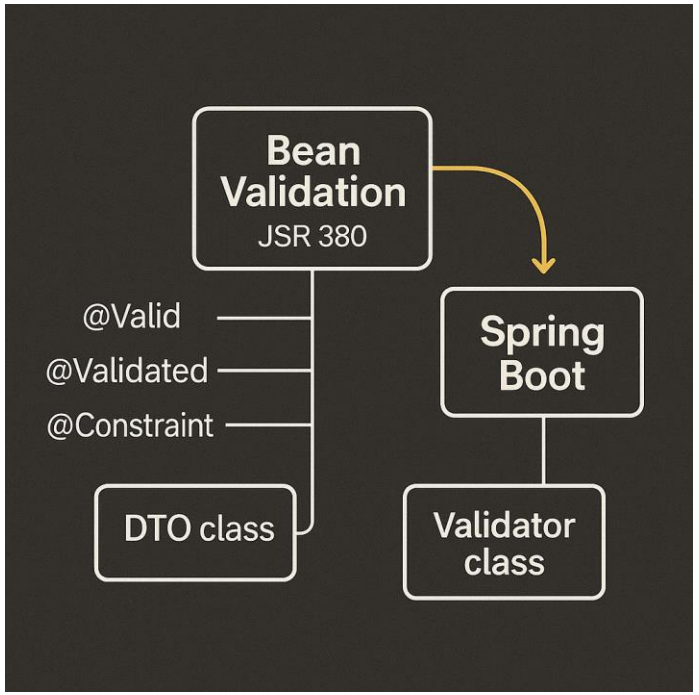
• ¿Cómo funciona Bean Validation?

Cuando aplicas **@Valid** a un parámetro **@RequestBody** en tu controlador de Spring Boot, si la validación falla (ej: un campo **@NotBlank** está vacío), Spring realiza automáticamente dos cosas:

1. Lanza una excepción específica: **MethodArgumentNotValidException**.
2. Por defecto, Spring Boot tiene un manejador interno que atrapa esta excepción y devuelve una respuesta **HTTP** con el código de estado **400 Bad Request**.

Conclusión:

El cliente recibe un 400, lo cual es el código de error semánticamente correcto. Sin embargo, el cuerpo del mensaje de error suele ser el JSON o XML genérico de Spring, que puede ser muy verboso y difícil de consumir por parte del cliente.



- **Documentación**

Bean Validation

Video de Apoyo

- **Documentación**

Flyway

Video de Apoyo

Liquibase

Documentación Oficial Liquibase

Video de Apoyo

03

Actividades

• ACTIVIDAD PRÁCTICA 1

Usa el proyecto **Proyecto Hospital Vida y Meditación**

Modifica las respuestas de los Endpoints ya construidos por ResponseEntity e integra Bean Validation.

04

¡MUCHAS GRACIAS!

DuocUC®

CERCANÍA. LIDERAZGO. FUTURO.

duoc.cl

7 AÑOS ACREDITADO

Comisión Nacional de Acreditación CNA-Chile

DESDE AGOSTO 2017 HASTA AGOSTO 2024.
DOCENCIA DE PREGRADO. GESTIÓN INSTITUCIONAL. VINCULACIÓN CON EL MEDIO.